

## Group 13: Raghav Jain and Pavan Desai

Please implement a visualization and optimization tool for printing in Cyan, Magenta, Yellow, Black (CMYK).

Here are the details:

- Read an arbitrary image, search for images that are suitable for demonstration purposes, i.e. have a high saturation or dark shades.
- Perform a pixel-wise transformation to the CMYK color space using only for-loops and if-clauses or using numpy (i.e. matrix operations).
- Display the 4 components (CMYK) individually as gray scale and save them as individual images
- Provide an estimation of the ink consumption for each pixel in another image (i.e.  $C+Y+M+K$ , may be up to 400%).
- Limit the ink consumption to a maximum value (i.e. 250%, 300%)
- Transform the image back to RGB and provide a comprehensive analysis of possible transformation losses, at least a difference image (potentially scaled to show errors), preferably with Delta-E analysis. A dedicated library may be used for the latter: colormath.

### Abstract:

This topic is about creating a visualisation and optimization tool for CMYK-based printing using computer vision and image processing in Python.

Here an RGB image is taken as input with spiked up saturation and was converted to CMYK to color space using numpy operations. Each CMYK component was taken in a grayscale ink separation plate and served as an independent image. The total ink consumption per pixel was computed as the sum of all 4 CMYK channels which results in calculation Total ink coverage (TIC). These values reach up to the total of 400% if we'll consider the maximum normalized value of CMYK as  $(1 + 1 + 1 + 1) * 100$ . Therefore, excessive ink deposition can lead to print quality issues, bleeding of ink and slow drying. Ink limitations are implemented to keep them under the practical threshold of 250% to 300%.

After ink limitation, these CMYK images are converted back to RGB for visual comparison. To evaluate the loss that is beared while transformation, CIEDE2000 Delta E operations are performed. The result afterwards, preserve the excessive CMYK usage while preserving the visible appearance of the image under the threshold of 250% and 300%.

# Methodology:

6 major steps have been followed in the methodology:

## 1. Image acquisition and Preprocessing:

A highly saturated photo from iPhone 16 has been clicked under the normal conditions and has been edited to spike up the saturation using Adobe Photoshop. In this process an HEIC image was converted to JPG. Later the RGB photo was taken and was normalized to value of 0-1. Since, most of the color models follow 0 - 1 and not 0 - 255.

## 2. RGB to CMYK Conversion:

The normalized RGB image was converted into the CMYK image using vectorized NumPy equations. For every pixel, the black point has been estimated from the maximum RGB intensity later followed on by conversion into Cyan, Magenta and Yellow channels.

RGB To CMYK Conversion:

$K = 1 - \max(R, G, B)$  Here, Black point is calculated.

$$C = (1 - R - K) / (1 - K)$$

$$M = (1 - G - K) / (1 - K)$$

$$Y = (1 - B - K) / (1 - K)$$

This is used to calculate CMY Channels.

## 3. CMYK Channel Visualization:

Total Ink Coverage (TIC) per pixel was calculated as:

$$TIC = (C + M + Y + K) * 100$$

Pixels exceeding the threshold values were optimized using a proportional CMY reduction strategy while preserving the black channel as much as possible. Limits that were considered are 250 % and 300%.

## 4. CMYK to RGB Re-Conversion:

The CMYK channels were transformed back into the RGB space. This will help in having a direct comparison with the initial image that was provided in the starting.

## 5. Transformational Loss Analysis:

Pixel wise RGB mapping and Delta E 2000 analysis were performed.

For the Delta E analysis, both original and reconstructed RGB images were converted to LAB color space and the CIEDE2000 Formula was used to compute resolution based perceptual mapping.

$$\Delta E_{2000} = \sqrt{\left(\frac{\Delta L'}{K_L S_L}\right)^2 + \left(\frac{\Delta C'}{K_C S_C}\right)^2 + \left(\frac{\Delta H'}{K_H S_H}\right)^2 + R_T \left(\frac{\Delta C'}{K_C S_C}\right) \left(\frac{\Delta H'}{K_H S_H}\right)}$$

- **Sl** – Defines how bright or Dark
- **Sc** - Adjusts the chroma difference(Vivid or dull)
- **Sh** - Adjust Hue difference (Which color family it belongs to like R,G,B)
- **Delta E** - There is a huge difference in the original and reconstructed image.
- **Delta e** – Reconstructed image is closer to the original image
- **Rt** - a correction term for certain hue regions, especially around blue.

## Methodology:

An RGB image was converted to CMYK and 4 grayscale channels were created for Cyan, Magenta, Yellow and Black.

The total ink consumption analysis revealed highly saturated regions that demand more CMYK levels.

After the execution of the ink limits 250 % and 300%, the heatmaps were created that showed that no pixel was exceeding the given thresholds. This demonstrates successful ink deposition.

The reconstructed RGB image showed that the 300% limited version showed very less changes and was visually super identical to the original image, whereas in the 250% limited version, there were slight color deviations in the most ink - intensive regions such as clouds and trees.

CIEDE2000 Delta-E helped to give these results:

**250% limit:** Avg  $\Delta E = 0.0054$ , Max  $\Delta E = 2.12$ , P95  $\Delta E = 0.0000$

**300% limit:** Avg  $\Delta E \approx 0.0000$ , Max  $\Delta E \approx 0.00$ , P95  $\Delta E \approx 0.0000$

These values indicate that under the 250% optimization, only a very few small fractions of pixels experienced a change while the 95% of the image stayed unchanged. Under the 300%, the image

experienced no measurable loss.

Overall, the project demonstrated CMYK implementation can significantly control the amount of ink that is deposited while maintaining the strong visual fidelity, especially when moderate ink limitations (i.e 300%) are implemented.

## References and Citations:

- RGB image:  
<https://kharshit.github.io/blog/2020/01/17/color-and-color-spaces-in-computer-vision>
- RGB and CMYK: Westland S & Cheung V, 2012. RGB Systems, cmyk Systems in Handbook of Visual Display Technology
- CIE2000 Formula:  
<https://www.konicaminolta.com/instruments/knowledge/color/part4/09.html>
- <https://arrowpix.in/rgb-to-cmyk-converter>
- Acquisition of Color Reproduction Technique based on Deep Learning Using a Database of Color-converted Images in the Printing Industry Ikumi Hirose† and Ryosuke Yabe†
- <https://colourlab.readthedocs.io/en/latest/> - Website
- <https://github.com/ifarup/colourlab> - Github
- <https://techkon.datacolor.com/demystifying-the-cie-delta-e-2000-formula/#:~:text=The%20CIE%20dE2000%20formula%20takes,along%20with%20the%20weighting%20functions>
- Taylor, G. (2017), python-colormath Documentation, Release 2.2.0.  
[https://python-colormath.readthedocs.io/\\_/downloads/en/2.2.0/pdf/](https://python-colormath.readthedocs.io/_/downloads/en/2.2.0/pdf/)